KIROVAULT

Build, Installer & Release Guide

Repeatable Windows development, packaging, validation and release process.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Developer setup

• Install Node.js/npm, Rust toolchain, Windows C++ build tools and WebView2 prerequisites as required by current Tauri docs.

• Create React + TypeScript with Vite; add Tauri 2; initialize app configuration.

• Run development build and validate native command boundary before adding broad filesystem features.

• Pin dependencies and commit lockfiles.

Build workflow

• Install dependencies from the lockfile and run lint/type-check/unit tests.

• Build frontend assets; run Tauri native build and resolve packaging warnings.

• Verify app identity, icon, version, permissions/capabilities and bundled assets.

• Build release artifacts on a clean, documented Windows environment.

Release checklist

• Verify installer and uninstaller, app icon, title, About screen and version.

• Test offline startup, drive navigation, preview fallback and settings persistence.

• Review bundled FFmpeg/PDF/media dependencies and required license notices.

• Prepare release notes, known limitations, checksums and rollback plan.

• Avoid logging sensitive paths/content in diagnostic output.

Versioning and updates

• Use semantic versioning policy and document breaking changes.

• Test upgrades from previous supported version and preserve user settings/index safely.

• If updates are introduced later, verify signing, transport and rollback behavior.

• Keep a reproducible release checklist and archive the build metadata.

Troubleshooting

• WebView2/runtime issue: check official Tauri Windows prerequisites.

• Native compilation issue: verify Rust target and C++ build tools.

• Preview mismatch: distinguish container support from codec/decoder support.

• Permission error: explain blocked path; do not auto-elevate.

References

• Tauri prerequisites: https://v2.tauri.app/start/prerequisites/

• Tauri bundling: https://v2.tauri.app/distribute/

• Node.js downloads: https://nodejs.org/en/download

• Rust installation: https://www.rust-lang.org/tools/install

• Microsoft WebView2: https://developer.microsoft.com/en-us/microsoft-edge/webview2/

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.